

End-to-end deep learning project for detecting melanoma diseases.

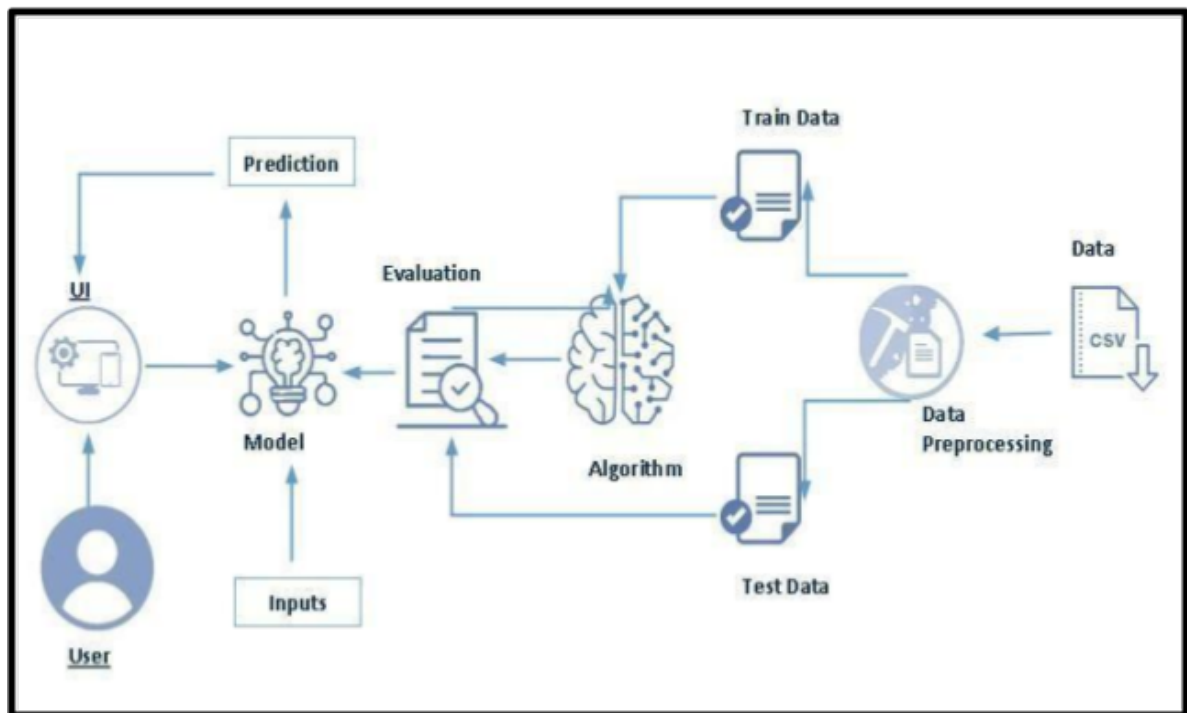
The objective of this project is to develop an end-to-end deep learning solution for detecting melanoma disease. The proposed solution involves the use of convolutional neural networks (CNNs) to extract relevant features from the input images and classify them into one of the nine categories.

Detecting melanoma disease as soon as possible is crucial because it can have significant impacts on our skin health. Early detection allows humans to take required action to prevent the spread of the disease. Here are some reasons why detecting melanoma disease early is essential:

1. **Improved Prognosis:** Early detection of melanoma allows for timely treatment, which can lead to better outcomes and a higher chance of successful recovery. Melanoma, when caught early, is often curable with simple surgical interventions.
2. **Reduced Mortality:** Early detection can help prevent the spread of melanoma to other parts of the body, which is a significant factor in reducing mortality rates associated with advanced melanoma.
3. **Less Aggressive Treatment:** Early-stage melanoma may require less aggressive treatments compared to advanced stages. Avoiding extensive treatments, such as chemotherapy or radiation, can improve the patient's quality of life and reduce healthcare costs.
4. **Scalability:** Once trained, deep learning models can be deployed on a large scale, making it possible to reach remote or underserved areas where access to specialized healthcare may be limited.
5. **Reduced Cost and Time:** Automation through deep learning can speed up the diagnosis process, leading to quicker treatment decisions and potentially lowering healthcare costs associated with delayed or misdiagnosed cases.

Let us look at the Technical Architecture of the project.

Technical Architecture:



Project Flow:

- The user interacts with the UI to upload the file.
- Uploaded input is analyzed by the model which is integrated/developed by you.
- Once the model analyzes the input the prediction is showcased on the

UI. To accomplish this, we have to complete all the activities listed below,

- Define Problem / Problem Understanding
 - 0 Specify the business problem
 - o Business requirements
 - o Literature Survey.
 - o Social or Business Impact.
- Data Collection & Preparation
 - o Collect the dataset
 - o Data Preparation
- Exploratory Data Analysis
 - 0 Descriptive statistical
 - o Visual Analysis
- Model Building
 - o Building a model.
 - 0 Training the model.
 - o Testing the model
- Model Deployment
 - 0 Save the best model
 - o Integrate with Web Framework
- Project Demonstration & Documentation
 - 0 Record explanation Video for project end to end solution

- Project Documentation-Step by step project development procedure

Prior Knowledge:

To complete this project, you must require following software's , concepts and packages

- VS Code :
 - Refer to the link below to download VS Code.
 - Link : <https://code.visualstudio.com/download>
- Create Project:
 - Open VS Code.
 - Create a Folder and name it "Melanoma_Detection" .
- Deep Learning Concepts
 - CNN: <https://towardsdatascience.com/basics-of-the-classic-cnn-a3dce1225add>
 - Sequential Models :
<https://towardsdatascience.com/coding-a-convolutional-neural-network-cnn-using-keras-a-s-sequential-api-ec5211126875>
- Web concepts:
 - Get the gist on streamlit :
<https://www.geeksforgeeks.org/a-beginners-guide-to-streamlit/>

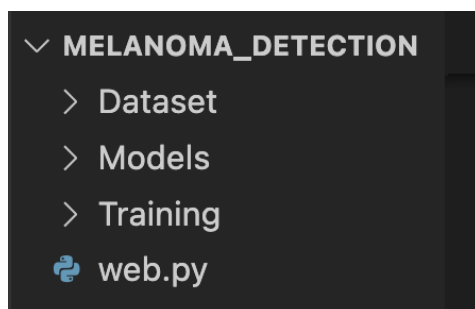
Project Objectives:

By the end of this project you will:

- know fundamental concepts and techniques of Convolutional Neural Network.
- Gain a broad understanding of image data.
- Knowhow to pre-process/clean the data using different data preprocessing techniques.
- Know how to build a web application using the Streamlit framework.

Project Structure:

Create the Project folder which contains files as shown below



- We are building a Streamlit application which needs a python script web.py for a website.
- Model folder contains your saved models.
- The Training folder contains your data and code for building/training/testing the model.

Milestone 1: Define Problem / Problem Understanding

Activity 1: Specify the business problem

Refer Project Description

Activity 2: Business requirements

Here are some potential business requirements for an ecommerce product delivery estimation predictor using deep learning:

1. Accurate prediction: The predictor must be able to accurately predict the type of skin cancer. The accuracy of the prediction is crucial for doctors, citizens, and other stakeholders to make informed decisions on the production.
2. User-friendly interface: The predictor must have a user-friendly interface that is easy to navigate and understand. The interface should present the results of the predictor in a clear and concise manner to enable doctors and other stakeholders to make informed decisions.
3. Scalability: The predictor must be able to scale up based on the prediction from our product. The model should be able to handle any size of data without compromising on its accuracy or efficiency.

Activity 3: Literature Survey (Student Will Write)

A literature survey would involve researching and reviewing existing studies, articles, and other publications on the topic of project. The survey would aim to gather information on current systems, their strengths and weaknesses, and any gaps in knowledge that the project could address. The literature survey would also look at the methods and techniques used in previous projects, and any relevant data or findings that could inform the design and implementation of the current project.

Activity 4: Social or Business Impact.

1. Improved potato crop yields: The potato leaf disease predictor can help farmers optimize their potato crop yields by providing accurate results. This can lead to increased crop yields and better crop quality, which can positively impact the income of farmers and the availability of good quality potatoes for consumers.
2. Reduction in production costs: The predictor can also help farmers reduce production costs by providing insights into optimal planting. This can lead to better resource allocation, reduced waste, and increased profitability for farmers.

Milestone 2: Data Collection & Preparation

DL depends heavily on data. It is the most crucial aspect that makes algorithm training possible. So, this section allows you to download the required dataset.

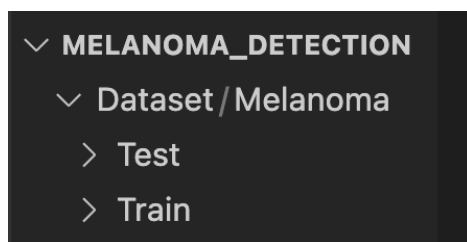
Activity 1: Collect the dataset.

There are many popular open sources for collecting the data. Eg: kaggle.com, UCI repository, etc.

This data is downloaded from kaggle.com. Please refer to the link given below to download the dataset.

Link : <https://www.kaggle.com/datasets/nodoubttome/skin-cancer9-classesisic>

Make sure your directory looks like below



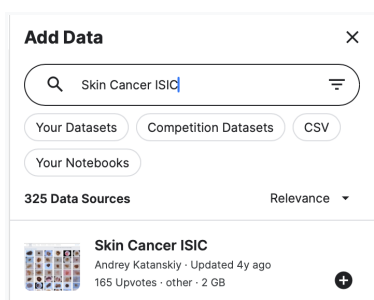
As the dataset is downloaded. Let us read and understand the data properly with the help of some visualization techniques and some analyzing techniques.

Note: We'll run all our codes on kaggle as the data is huge and we'll also perform parallel processing on mode using two GPUs.

Activity 1.1: Importing data and configuring GPUs.

Create a new notebook on kaggle and click on "+ Add Data" in the data section.

Search for Skin Cancer ISIC click on the "+" mark to add data .



We've imported the required data, now we'll initiate the GPU's.

Go to Notebook options and set as shown below.

Notebook options ^

ACCELERATOR

GPU T4 ×2

▼

Quota: 05:52 / 30 hrs

LANGUAGE

Python

▼

PERSISTENCE

No persistence

▼

ENVIRONMENT

Pin to original environment (2023-07-10)

▼

You won't get new packages, but your code is less likely to break. What is a notebook environment? ⓘ

Activity 1.2: Set ImageSize, BatchSize and Channels.

Set the values as shown below

```
batch_size = 32
img_height = 180
img_width = 180
channels = 3
```

Activity 1.2: Read the Dataset

Our dataset contains .jpg images. We can read the dataset with the help of tensorflow.

In tensorflow we have a function called `image_dataset_from_directory()` to read the dataset. We've already been given the divided test/train dataset.

Activity 2: Data Preparation

As we have understood how the data is, let's pre-process the collected data.

The download data set is not suitable for training the deep learning model. We have to extract the class names of the data and let's try to visualize the data with labels.

- Extracting class names.
- Visualizing the data

Activity 2.1: Extracting class names

- Let's find the class names of our dataset first. To find the class names of the dataset, we can use `.class_names`.

```
## Write your train dataset here
## Note use seed=123 while creating your dataset using tf.keras.preprocessing.image_dataset_from_directory
## Note, make sure your resize your images to the size img_height*img_width, while writing the dataset
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    r'/kaggle/input/skin-cancer9-classes/sic/Skin cancer ISIC The International Skin Imaging Collaboration/Train/',
    shuffle = True ,
    image_size = (img_height, img_width) , # image_size =(180,180,3)
    batch_size = batch_size # 32
)
```

```
train_ds.class_names
```

```
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    r'/kaggle/input/skin-cancer9-classesisic/Skin cancer ISIC The International Skin Imaging Collaboration/Test/',
    shuffle = True ,
    image_size = (img_height, img_width) , # image_size = (180,180,3)
    batch_size = batch_size                # 32
)
```


Activity 2.2: Visualizing the data

We have created batches of the data and we can access the batch by .take() function

```
for image_batch , label_batch in val_ds.take(1) :  
    print('this is one batch shape :',image_batch.shape) # (batch_size,img_height,img_width,chane1ls)  
    print('this is lable shape :',label_batch.shape)
```

```
this is one batch shape : (32, 180, 180, 3)  
this is lable shape : (32,)
```

```
for image_batch , label_batch in train_ds.take(1):  
    print(image_batch.numpy())  
    print('\n')  
    print('convergt the lable in numeric : ',label_batch.numpy())
```

The above code is to visualize the data in numeric format and the below code is to visualize the data in image format

```
import matplotlib.pyplot as plt  
  
### your code goes here, you can use training or validation data to visualize  
  
plt.figure(figsize=(15,20))  
for image_batch , labels_batch in train_ds.take(1):  
    print(image_batch.shape)  
    print(labels_batch.numpy())  
    for i in range(32):  
        plt.subplot(8,4,i+1)  
        plt.imshow(image_batch[i].numpy().astype('uint8'))  
        plt.title(class_names[labels_batch[i]])  
        plt.axis('off')
```

- Image batch contains images and label batch contains labels of the particular image.
- Above code subplots 32 images from the 1st batch of the extracted dataset.

Milestone 3: Exploratory Data Analysis

Activity 1: Descriptive statistical

Descriptive analysis is to study the basic features of data with the statistical process. With this describe function we can understand the unique, top and frequent values of categorical features. And we can find mean, std, min, max and percentile values of continuous features. Which are not suitable for our dataset.

Activity 2: Visual analysis

Visual analysis is the process of using visual representations, such as charts, plots, and graphs, to explore and understand data. It is a way to quickly identify patterns, trends, and outliers in the data, which can help to gain insights and make informed decisions.

This is not needed for our current dataset as it is an image dataset. Visual analysis is mostly used for numerical data.

Activity 3: Splitting data into train and test and validation sets

Now let's split the Dataset into train, test and validation sets. First split the dataset into train and test sets.

The split will be in 8:1:1 ratio : train : test : validation respectively

Activity 4: Optimizing, Resize, Rescale and Augmentation of the data

```
#optimizing
AUTOTUNE = tf.data.experimental.AUTOTUNE
train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
```

```
# resize and rescale the value
resize_and_rescale = tf.keras.Sequential([
    tf.keras.layers.experimental.preprocessing.Resizing(img_height, img_width),
    tf.keras.layers.experimental.preprocessing.Rescaling(1.0/255)
])
```

```
# data augmentation
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.experimental.preprocessing.RandomFlip('horizontal_and_vertical'),
    tf.keras.layers.experimental.preprocessing.RandomRotation(0.3)
])
```

- Prefetching and Caching saves a lot of time in accessing the data from disks. Refer to this [link](#) for better understanding.
- Resize and Rescale is used to maintain the uniformity among the data.
- Augmentation helps you to increase the amount of data and helps your model to learn better.

Milestone 4: Model Building

Activity 1: Building model 1

Now our data is ready and it's time to build the model.

Here we are going to build our model layer by layer using .Sequential() from keras.

Let's go !

```

from tensorflow.keras.layers import Dense, Dropout, Activation, Flatten, Conv2D, MaxPooling2D, BatchNormalization
input_shape = (batch_size, img_height, img_width, channels)
n_classes = 9

def create_model():
    # create CNN model

    model = Sequential()

    model.add(resize_and_rescale)

    model.add(data_augmentation)

    model.add(Conv2D(32, kernel_size=(3,3), padding='same', activation='relu', input_shape=input_shape))

    model.add(Conv2D(64, kernel_size=(3,3), padding='same', activation='relu'))

    model.add(MaxPooling2D(2,2))

    model.add(Conv2D(64, kernel_size=(3,3), padding='same', activation='relu'))

    model.add(MaxPooling2D(2,2))

    model.add(Flatten())

    model.add(Dense(32, activation='relu'))

    model.add(Dense(64, activation='relu'))

    model.add(Dropout(0.10))

    model.add(Dense(n_classes))

    return model

```

The next step would be to train the model on two GPUs parallelly.
For that we need to develop a mirrored strategy and run our model within that scope.

```

strategy = tf.distribute.MirroredStrategy()
print("Total GPUs : ", strategy.num_replicas_in_sync)

```

Total GPUs : 2

After creating the strategy we'll train the model as shown below

```

with strategy.scope() as scope:
    model = create_model()
    model.compile(optimizer='adam', loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=['accuracy'])
    model.build(input_shape)
    model.summary()
    epochs = 5
    history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=epochs
    )

```

Now, let's visualize the model's performance.

```

acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()

```

Activity 2: Building model 2

After you have analyzed the model fit history for presence of underfit or overfit, choose an appropriate data augmentation strategy.

```

data_augmentation = keras.Sequential(
    [
        layers.experimental.preprocessing.RandomFlip("horizontal",
                                                    input_shape=(img_height,
                                                                    img_width,
                                                                    3)),
        layers.experimental.preprocessing.RandomRotation(0.1),
        layers.experimental.preprocessing.RandomZoom(0.1)
    ]
)

```

visualize how your augmentation strategy works for one instance of training image.

```

plt.figure(figsize=(12, 12))
for images, labels in train_ds.take(1):
    for i in range(9):
        augmented_images = data_augmentation(images)
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(augmented_images[i].numpy().astype("uint8"))
        plt.title(class_names[labels[i]])
        plt.axis("off")

```

Let's build the second model and hope for better results.

```
def create_model():
    ## You can use Dropout layer if there is an evidence of overfitting in your findings
    ## Your code goes here
    model = Sequential()
    model.add(data_augmentation)
    model.add(resize_and_rescale)
    model.add(Conv2D(16, kernel_size=(3,3), padding='same', activation='relu'))
    model.add(MaxPooling2D(2,2))
    model.add(Conv2D(32, kernel_size=(3,3), padding='same', activation='relu'))
    model.add(MaxPooling2D(2,2))
    model.add(Conv2D(64, kernel_size=(3,3), padding='same', activation='relu'))
    model.add(MaxPooling2D(2,2))
    model.add(Dropout(0.2))
    model.add(Flatten())
    model.add(Dense(128, activation='relu'))
    model.add(Dense(n_classes))

    return model
```

Notebook editor cells

Compile and train the model.

```
with strategy.scope() as scope:
    model = create_model()
    model.compile(optimizer='adam', loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), metrics=['accuracy'])
    model.build(input_shape)
    model.summary()
    epochs = 5
    history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=epochs
    )
```

Visualizing the results

```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```

You should be getting the almost same results as compared to model 1.

Activity 3: Building model 3

In model 3, we'll use transfer learning and different augmenting technique for the better result.

For that, first we need to move the data from the input folder of kaggle to the output folder of kaggle.

```
import shutil

source_directory = "/kaggle/input/skin-cancer9-classesisic/Skin cancer ISIC The International Skin Imaging Collaboration/"
destination_directory = "/kaggle/working/melanoma-classes"

# Use shutil.copytree() to copy the entire directory and its contents
try:
    shutil.copytree(source_directory, destination_directory)
    print("Directory copied successfully.")
except OSError as e:
    print(f"Error: {e}")
```

Directory copied successfully.

```
data_dir_train = pathlib.Path("/kaggle/working/melanoma-classes/Train")
```

```
image_count_train = len(list(data_dir_train.glob('*/*.jpg')))
print(image_count_train)
```

2239

```
#plot number of images in each Class
count=[]
for name in class_names:
    count.append(len(list(data_dir_train.glob(name+'/*.jpg'))))
```

```
count
```

[114, 376, 95, 438, 357, 462, 77, 181, 139]

There are a total of 2239 images divided into 9 classes abruptly. Let's visualize them.

```
#plot number of images in each Class
count=[]
for name in class_names:
    count.append(len(list(data_dir_train.glob(name+'/*.jpg'))))
plt.figure(figsize=(25,10))
plt.bar(class_names,count)
```

You'll get to see an imbalanced plot.

So, now we'll augment the data using the package called "Augmentor".

```
!pip install Augmentor
```

```
path_to_training_dataset=str(data_dir_train)+'/'
import Augmentor
for i in class_names:
    p = Augmentor.Pipeline(path_to_training_dataset + i)
    p.rotate(probability=0.7, max_left_rotation=10, max_right_rotation=10)
    p.sample(650) ## We are adding 500 samples per class to make sure that none of the classes are sparse.
```

The above code will take some time and adds the augmented data to your actual data and makes it a balanced data.

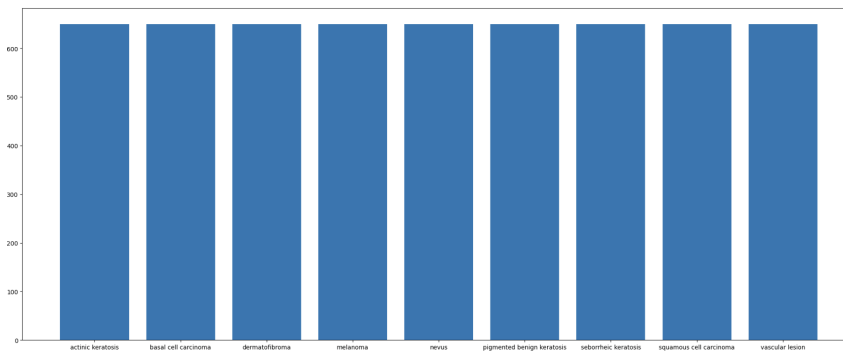
```
image_count_train = len(list(data_dir_train.glob('*output/*.jpg')))
print(image_count_train)
```

The count should be **5850**.

Let's see if the data is balanced or not.

```
# Check the distribution of data again.
count=[]
for name in class_names:
    count.append(len(list(data_dir_train.glob(name+'*output/*.jpg'))))
plt.figure(figsize=(25,10))
plt.bar(class_names,count)
```

<BarContainer object of 9 artists>



Now, Let's create a dataframe which includes "Path" and "Label" of every image in the dataset.

```
import os
from glob import glob
path_list_new = [x for x in glob(os.path.join(data_dir_train, '*', 'output', '*.jpg'))]
lesion_list_new = [os.path.basename(os.path.dirname(os.path.dirname(y))) for y in glob(os.path.join(data_dir_train, '*', 'output', '*.jpg'))]
dataframe_dict_new = dict(zip(path_list_new, lesion_list_new))
```

```
# Get Existing images in Dataframe
path_list=[]
lesion_list=[]
for name in class_names:
    for file in data_dir_train.glob(name+'/*.jpg'):
        path_list.append(str(file))
        lesion_list.append(name)

dataframe_dict_original=dict(zip(path_list,lesion_list))
original_df=pd.DataFrame(list(dataframe_dict_original.items()),columns=['Path', 'Label'])
```

We've created a balanced dataset and now we'll preprocess the train and test data using

tensorflow.

```
train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    data_dir_train,
    seed=123,
    validation_split = 0.2,
    subset = 'training',
    image_size=(img_height, img_width),
    batch_size=batch_size)
```

Found 8089 files belonging to 9 classes.
Using 6472 files for training.

```
val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    data_dir_train,
    seed=123,
    validation_split = 0.2,
    subset = 'validation',
    image_size=(img_height, img_width),
    batch_size=batch_size)
```

Found 8089 files belonging to 9 classes.
Using 1617 files for validation.

```
## your code goes here
AUTOTUNE = tf.data.experimental.AUTOTUNE

train_ds = train_ds.cache().shuffle(1000).prefetch(buffer_size=AUTOTUNE)
val_ds = val_ds.cache().prefetch(buffer_size=AUTOTUNE)
```

Let's create a model using transfer learning and we'll be using the VGG16 model for transfer learning.

```

import tensorflow as tf
from tensorflow.keras.applications import vgg16
from tensorflow.keras.models import Model, Sequential
from tensorflow.keras import layers
import keras
def create_model_vgg():
    vgg = vgg16.VGG16(include_top=False, weights='imagenet',
                      input_shape=(180,180,3))

    output = vgg.layers[-1].output
    output = keras.layers.Flatten()(output)
    vgg_model = Model(vgg.input, output)

    vgg_model.trainable = False
    for layer in vgg_model.layers:
        layer.trainable = False
    model = Sequential()
    model.add(vgg_model)
    model.add(Dense(512, activation='relu', input_dim=(180,180,3)))
    model.add(Dropout(0.3))
    model.add(Dense(256, activation='relu'))
    model.add(Dropout(0.3))
    model.add(Dense(128, activation='relu'))
    model.add(Dropout(0.3))
    model.add(Dense(64, activation='relu'))
    model.add(Dropout(0.3))
    model.add(Dense(32, activation='relu'))
    model.add(Dropout(0.3))
    model.add(Dense(n_classes, activation='sigmoid'))

    return model

```

Defining scheduler callback function :

```

def scheduler(epoch, lr):
    if epoch < 35:
        return lr
    else:
        return lr * tf.math.exp(-0.1)

```

```

from tensorflow.keras.callbacks import LearningRateScheduler
schedule_callback = LearningRateScheduler(scheduler)

```

Training the model :

```

import tensorflow as tf
from tensorflow.keras.optimizers import Adam
with strategy.scope() as scope:
    model = create_model_vgg()
    ## your code goes here
    optimizer = Adam(learning_rate=0.0001)
    model.compile(optimizer=optimizer,
                  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=False),
                  metrics=['accuracy'])
    model.build(input_shape)
    model.summary()
    epochs = 40

    history = model.fit(
        train_ds,
        validation_data=val_ds,
        epochs=epochs,
        callbacks = schedule_callback
    )

```

Note

Milestone 5: Model Deployment

Activity 1: Model Evaluation

As we have the record of accuracy stored in the history variable. We can try visualizing the performance of our model.

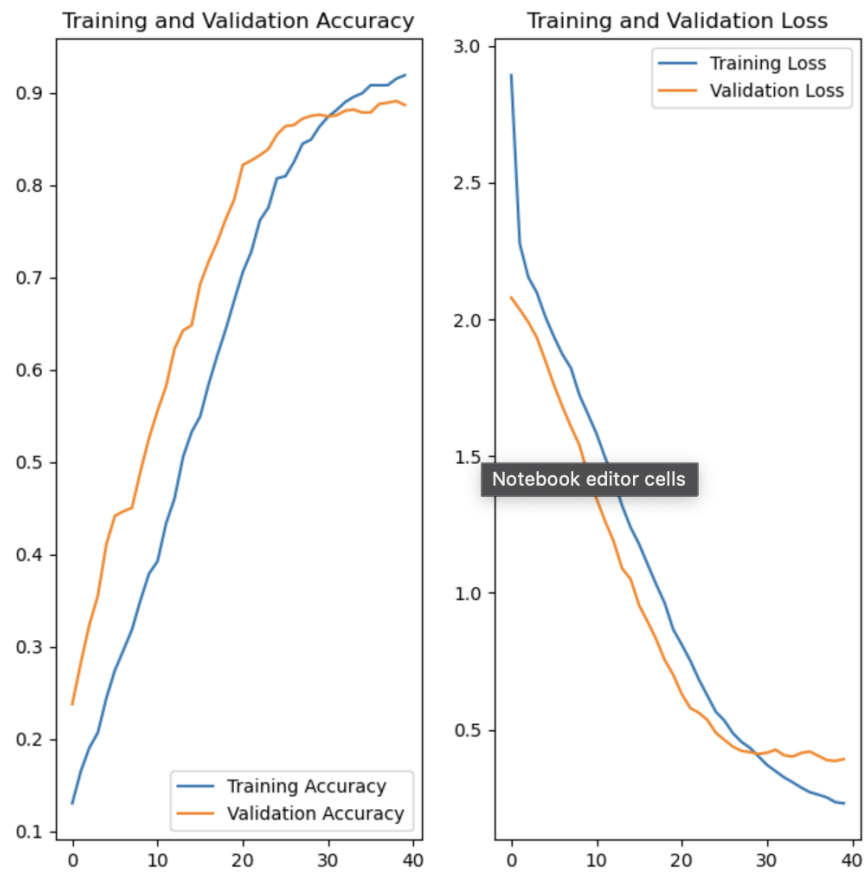
```
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']

loss = history.history['loss']
val_loss = history.history['val_loss']

epochs_range = range(epochs)

plt.figure(figsize=(8, 8))
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy')
plt.plot(epochs_range, val_acc, label='Validation Accuracy')
plt.legend(loc='lower right')
plt.title('Training and Validation Accuracy')

plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss')
plt.plot(epochs_range, val_loss, label='Validation Loss')
plt.legend(loc='upper right')
plt.title('Training and Validation Loss')
plt.show()
```



Your plot should look something like this.

Let's save this model.

Activity 2: Save the model

```
model.save("/kaggle/working/Melanoma_Detection6.h5")
```

This will allow us to save the .h5 format of the model.

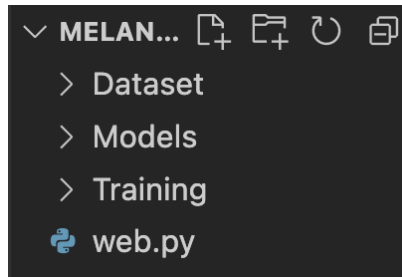
Activity 4: Integrate with Web Framework

In this section, we will be building a web application that is integrated to the model we built.

We will be using the streamlit package for our website development.

Streamlit is a free and open-source framework to rapidly build and share beautiful machine learning and data science web apps.

Activity 4.1: Create a web.py file and import necessary packages:



```
import streamlit as st
import tensorflow as tf
from PIL import Image, ImageOps
import numpy as np
from io import BytesIO
import tensorflow_hub as hub
```

NOTE : If you get any error like “streamlit not found”. You have to make sure streamlit is installed on your system.

- Running the command “pip install streamlit” would do that job for you.

Activity 4.2: Defining class names and loading the model:

```
st.cache(allow_output_mutation=True)
CLASS_NAMES = ['actinic keratosis',
               'basal cell carcinoma',
               'dermatofibroma',
               'melanoma',
               'nevus',
               'pigmented benign keratosis',
               'seborrheic keratosis',
               'squamous cell carcinoma',
               'vascular lesion']

def load_model():
    model=tf.keras.models.load_model('./Models/Melanoma_Detection7.h5',
                                     custom_objects={'KerasLayer':hub.KerasLayer})
    return model
model = load_model()
```

Activity 4.3: Accepting the input from user and prediction

```
st.write("""# Melanoma Detection""")

file = st.file_uploader("Please upload an image file", type=["jpg", "png"])

def import_and_predict(image_data, model):
    size = (180,180)
    image = ImageOps.fit(image_data, size, Image.ANTIALIAS)
    image = np.asarray(image)
    img_reshape = np.expand_dims(image,0)
    prediction = model.predict(img_reshape)
    return prediction
```

- st.write() function will write the title for your website.
- In the st.file_uploader() function, 'type' parameter helps us to restrict the type of input data.

```
if file is None:
    st.text("Please upload an image file")
else:
    image = Image.open(file)
    st.image(image, use_column_width=True)
    predictions = import_and_predict(image, model)
    index = np.argmax(predictions[0])
    predicted_class = CLASS_NAMES[index]
    confidence = np.max(predictions[0])
    st.write(predicted_class)
    st.write(confidence)
    print(
        "This image most likely belongs to {} with a {:.2f} percent confidence."
        .format(CLASS_NAMES[np.argmax(confidence)], 100 * np.max(confidence))
    )
```

- The above lines of code helps us to take the input from the user and predict the results.
- The website prompts you the text until you upload a file.
- Once you upload an image it starts predicting the label that it should be belonging to.

To run your website go to your terminal with your respective directory and run the command :

- "streamlit run web.py"

You can now view your Streamlit app in your browser.

Local URL: `http://localhost:8501`

Network URL: `http://192.168.1.11:8501`

For better performance, install the Watchdog module:

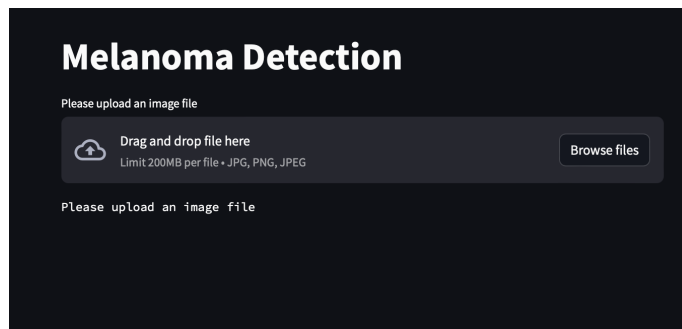
```
$ xcode-select --install
```

```
$ pip install watchdog
```

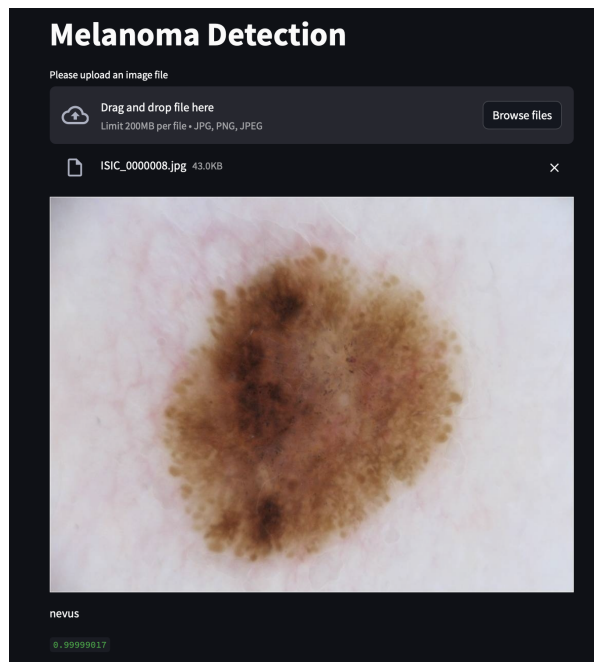
On successful execution you'll get to see this in your terminal.

HOW DOES YOUR WEBSITE LOOK LIKE ?

- Before uploading the picture :



- After uploading the picture :



Milestone 6: Project Demonstration & Documentation

Below mentioned deliverables to be submitted along with other deliverables.

Activity 1: - Record explanation Video for project end to end solution.

Activity 2: - Project Documentation-Step by step project development procedure.

Create a document as per the template provided.

